Sandi Bot Desktop — Production Roadmap & Cursor Build Guide

> **AI Solutions Architect:** Dr. Data — Decision Intelligence
> **Client:** Sandy Stahl — Franchise Coach
> **Product:** Airgapped Coaching Intelligence Desktop Application
> **POC Reference:** https://github.com/Zmugha1/Sandi_Bot_Coaching_Intelligence
> **Production Repo:** https://github.com/Zmugha1/Sandi_Bot_Desktop

---

⚠️ Critical Rules Before Touching Any Code

1. **Never modify the POC repo.** `Sandi_Bot_Coaching_Intelligence` is frozen. It is the live demo on Netlify. This production repo is separate.
2. **Never use `better-sqlite3`.** It is a Node.js native module incompatible with Tauri. Always use `tauri-plugin-sql` with SQLite.
3. **Never use `pdf-parse`.** Use `pdf-extract` (Rust crate) as primary, `lopdf` as fallback.
4. **Never upgrade Tailwind to v4.** Stay on Tailwind 3.4 — it matches the POC exactly.
5. **Never use Electron.** Tauri v2 only. Smaller installer, better performance, native Rust backend.
6. **Always test Rust changes with `cargo check` before `npm run tauri:dev`.**
7. **Every recommendation the system makes must be logged to the audit table.** Governance is non-negotiable.

Tech Stack — Complete

Layer	Technology	Notes
UI Framework	React 19 + TypeScript	Preserved from POC
Build Tool	Vite 5	Preserved from POC
Styling	Tailwind CSS 3.4	DO NOT upgrade to v4
Components	shadcn/ui + Radix UI	Preserved from POC
Charts	Recharts	Preserved from POC
Desktop Shell	Tauri v2 (Rust)	~5MB installer
IPC Bridge	@tauri-apps/api invoke()	TS → Rust commands
Database	SQLite via tauri-plugin-sql	Single .sqlite file
Search	SQLite FTS5	Built into SQLite
File System	tauri-plugin-fs	Native OS access
Folder Picker	tauri-plugin-dialog	Backup location UI

PDF Extraction	pdf-extract (Rust crate)	Primary text extractor
PDF Fallback	lopdf (Rust crate)	Secondary extractor
OCR (if needed)	tesseract-rs	Only for scanned PDFs
LLM Runtime	Ollama (local server)	localhost:11434
LLM Model (fast)	phi3:mini	2.3GB, simple docs
LLM Model (smart)	llama3.1:8b	4.7GB, complex docs
Knowledge Graph	Neo4j Community (local)	Phase 5+
Graph Embeddings	Node2Vec (Python script)	Phase 6C+

Folder Structure — Complete Production

```
Sandi_Bot_Desktop/
├── src/                        # React frontend (TypeScript)
│   ├── main.tsx                # Entry point
│   ├── App.tsx                 # Root component — 8 module router
│   ├── index.css               # Global styles + Tailwind + CSS vars
│   ├── modules/                # The 8 core UI modules
│   │   ├── ExecutiveDashboard.tsx
│   │   ├── ClientIntelligence.tsx
│   │   ├── PipelineVisualizer.tsx
│   │   ├── LiveCoachingAssistant.tsx
│   │   ├── PostCallAnalysis.tsx
│   │   ├── AdminStreamliner.tsx
│   │   ├── AuditTransparency.tsx
│   │   └── HowToUse.tsx
│   ├── services/               # Data access layer
│   │   ├── db.ts               # Database connection singleton
│   │   ├── clientService.ts    # Client CRUD operations
│   │   ├── documentService.ts  # Document storage + retrieval
│   │   ├── searchService.ts    # FTS5 knowledge base queries
│   │   ├── auditService.ts     # Audit log writes + reads
│   │   ├── backupService.ts    # Backup status + triggers
│   │   └── ollamaService.ts    # LLM parsing calls
│   ├── components/             # Shared UI components
│   │   ├── ErrorBoundary.tsx   # Module-level crash recovery
│   │   ├── StatusBar.tsx       # Backup status + Ollama status
│   │   └── LoadingState.tsx    # Async loading indicators
│   └── types/                  # TypeScript type definitions
│       ├── client.ts
│       ├── document.ts
│       └── audit.ts
```

```
│
├── src-tauri/                  # Rust backend
│   ├── Cargo.toml              # Rust dependencies
│   ├── tauri.conf.json         # Tauri v2 configuration
│   ├── capabilities/
│   │   └── default.json        # Tauri v2 permissions
│   ├── icons/                  # App icons (required for build)
│   │   ├── 32x32.png
│   │   ├── 128x128.png
│   │   ├── 128x128@2x.png
│   │   ├── icon.icns
│   │   └── icon.ico
│   └── src/
│       ├── main.rs             # Tauri entry point + command registry
│       ├── lib.rs              # Module exports
│       ├── database.rs         # SQLite init + schema
│       ├── pdf_parser.rs       # PDF text extraction
│       ├── file_watcher.rs     # Folder monitoring
│       ├── backup.rs           # Backup system
│       └── ollama.rs           # LLM HTTP client (Phase 7)
│
├── prompts/                    # LLM prompt templates (Phase 7)
│   ├── disc.txt
│   ├── you2.txt
│   ├── fathom.txt
│   └── vision.txt
│
├── scripts/                    # Utility scripts
│   └── train_embeddings.py     # Node2Vec training (Phase 6C)
│
├── package.json
├── vite.config.ts
├── tailwind.config.js
├── tsconfig.json
├── postcss.config.js
├── PRODUCTION_ROADMAP.md       # This file
└── README.md                   # Setup instructions
```

Sandy's Machine — Folder Structure

```
C:/Users/Sandy/
```

```
├── SandiBot/                    # App data root
│   ├── clients/                 # WATCHED FOLDERS
│   │   ├── DISC/                # Drop DISC PDFs here
│   │   ├── You2/                # Drop You 2.0 reports here
│   │   ├── Fathom/              # Drop call transcripts here
│   │   └── Vision/              # Drop vision documents here
│   ├── backups/                 # Auto-backup location
│   │   └── sandi-bot-2026-03-01.sqlite
│   └── database.sqlite          # Primary database
│
└── AppData/Local/
    ├── ollama/models/           # LLM models (one-time download)
    │   ├── phi3:mini            # 2.3GB
    │   └── llama3.1:8b          # 4.7GB
    └── Neo4j/                   # Knowledge graph (Phase 5+)
        └── data/
```

Database Schema — Complete

```sql
```

```sql
-- CLIENTS (core record)
CREATE TABLE IF NOT EXISTS clients (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT,
    phone TEXT,
    company TEXT,
    stage TEXT DEFAULT 'Initial Contact',
    disc_style TEXT,
    disc_scores TEXT,           -- JSON: {d:78, i:45, s:62, c:31}
    you2_statement TEXT,
    you2_dangers TEXT,          -- JSON array
    you2_opportunities TEXT,    -- JSON array
    tumay_data TEXT,            -- JSON
    vision_statement TEXT,
    readiness_identity INTEGER DEFAULT 3,
    readiness_commitment INTEGER DEFAULT 3,
    readiness_financial INTEGER DEFAULT 3,
    readiness_execution INTEGER DEFAULT 3,
    confidence INTEGER DEFAULT 50,
    recommendation TEXT DEFAULT 'NURTURE',  -- PUSH/NURTURE/PAUSE
    outcome TEXT,               -- ACTIVE/CONVERTED/STALLED/CLOSED
    notes TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- DOCUMENTS (imported files)
CREATE TABLE IF NOT EXISTS documents (
    id TEXT PRIMARY KEY,
    client_id TEXT NOT NULL,
    filename TEXT NOT NULL,
    document_type TEXT NOT NULL,  -- DISC/You2/Fathom/Vision
    file_path TEXT NOT NULL,
    raw_text TEXT,
    extracted_data TEXT,          -- JSON from LLM parser
    validation_passed BOOLEAN DEFAULT false,
    uploaded_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (client_id) REFERENCES clients(id)
);

-- SESSIONS (coaching call records)
CREATE TABLE IF NOT EXISTS sessions (
```

```sql
    id TEXT PRIMARY KEY,
    client_id TEXT NOT NULL,
    session_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    stage_before TEXT,
    stage_after TEXT,
    summary TEXT,
    objections_raised TEXT,      -- JSON array
    scripts_used TEXT,           -- JSON array
    outcome_signal TEXT,         -- PROGRESS/STALL/CONCERN
    next_action TEXT,
    FOREIGN KEY (client_id) REFERENCES clients(id)
);

-- AUDIT LOG (governance — every recommendation logged)
CREATE TABLE IF NOT EXISTS audit_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    client_id TEXT,
    action_type TEXT NOT NULL,    -- RECOMMENDATION/PARSE/SEARCH/BACKUP
    input_data TEXT,             -- What triggered this
    output_data TEXT,            -- What the system decided
    reasoning TEXT,              -- Why — this is the governance field
    model_used TEXT              -- deterministic/phi3/llama3.1
);

-- KNOWLEDGE BASE (FTS5 full-text search)
CREATE VIRTUAL TABLE IF NOT EXISTS knowledge_search
USING fts5(
    content,
    content_type,    -- CLEAR_QUESTION/PINK_FLAG/DISC_TIP/SCRIPT
    stage,
    client_id
);

-- BACKUP LOG
CREATE TABLE IF NOT EXISTS backup_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    backup_path TEXT,
    success BOOLEAN,
    error_message TEXT
);
```

IPC Commands — Complete Registry

Every TypeScript `invoke()` call must have a matching `#[tauri::command]` in `main.rs`.

Command Name	Rust Handler	Module Using It
greet	greet()	Test only
get_app_dir	get_app_dir()	Setup
initialize_database	initialize_database()	Setup
get_all_clients	get_all_clients()	Dashboard, Pipeline
get_client	get_client()	ClientIntelligence
create_client	create_client()	ClientIntelligence
update_client	update_client()	ClientIntelligence
delete_client	delete_client()	ClientIntelligence
search_knowledge	search_knowledge()	CoachingAssistant
get_audit_log	get_audit_log()	AuditTransparency
log_audit_entry	log_audit_entry()	All modules
parse_pdf	parse_pdf()	AdminStreamliner
start_file_watcher	start_file_watcher()	App startup
stop_file_watcher	stop_file_watcher()	App shutdown
create_backup	create_backup()	AdminStreamliner
get_backup_status	get_backup_status()	StatusBar
get_pipeline_data	get_pipeline_data()	PipelineVisualizer
save_session	save_session()	PostCallAnalysis
get_sessions	get_sessions()	ClientIntelligence
get_recommendation	get_recommendation()	Dashboard, Coaching
check_ollama	check_ollama()	StatusBar (Phase 7)
parse_with_llm	parse_with_llm()	AdminStreamliner (Phase 7)

PHASE 1 — Tauri Desktop Foundation

Goal: App launches as a real desktop window. All 8 modules render. IPC bridge works.
Investment: $2,500
Timeline: Week 1

Cursor Prompt — Phase 1

Copy this entire prompt into Cursor:

I am building a Tauri v2 desktop app called Sandi Bot.
I have the frontend React POC at: https://github.com/Zmugha1/Sandi_Bot_Coaching_Intelligence
I am building the production desktop version from scratch in this repo.

TECH STACK (do not deviate):
- Tauri v2 (NOT v1)
- React 19 + TypeScript + Vite 5
- Tailwind CSS 3.4 (do NOT upgrade to v4)
- tauri-plugin-sql v2 for SQLite (NOT better-sqlite3)
- tauri-plugin-fs v2 for file operations
- tauri-plugin-dialog v2 for folder picker

PHASE 1 TASKS — do these in order:

1. Fix file_watcher.rs:
   - Change `thread::spawn(move {` to `thread::spawn(move || {`
   - Replace dirs::home_dir() with std::env::var("USERPROFILE") on Windows
     and std::env::var("HOME") on other platforms

2. Fix backup.rs and database.rs:
   - Replace all tauri_plugin_sql::Builder::default().build() calls
     with tauri_plugin_sql::Database::load()

3. Fix pdf_parser.rs lopdf extraction:
   - Get raw content stream bytes from doc.get_page_content()
   - Parse parenthesized text tokens using regex \(([^)]+)\)
   - Return extracted text tokens joined with spaces

4. Copy all 8 modules from the POC repo into src/modules/:
   - ExecutiveDashboard.tsx
   - ClientIntelligence.tsx
   - PipelineVisualizer.tsx
   - LiveCoachingAssistant.tsx
   - PostCallAnalysis.tsx
   - AdminStreamliner.tsx
   - AuditTransparency.tsx
   - HowToUse.tsx
   Keep the exact same UI. Do not change visual design.

5. Replace App.tsx test harness with real app shell:
   - Sidebar navigation between all 8 modules
   - Same navigation style as the POC
   - Import all 8 modules

```
   - Modules still use sample data from POC (real data wired in Phase 2)
   - Add ErrorBoundary wrapper around each module

6. Create src/components/ErrorBoundary.tsx:
   - Catches errors per module
   - Shows friendly error card instead of blank screen
   - Logs error to console

7. Verify tauri.conf.json has capabilities/default.json referenced

8. Run cargo check in src-tauri/ and fix any remaining compile errors

ACCEPTANCE TEST:
- npm run tauri:dev launches a desktop window
- All 8 modules are navigable in the sidebar
- No TypeScript errors
- No Rust compile errors
- cargo check passes clean
```

Phase 1 Acceptance Checklist

```
[ ] npm run tauri:dev launches desktop window
[ ] All 8 modules render without blank screen
[ ] Sidebar navigation works between modules
[ ] No TypeScript errors in terminal
[ ] cargo check passes with no errors
[ ] ErrorBoundary exists and wraps each module
[ ] App.css deleted and not imported
[ ] kimi plugin not present in vite.config.ts
```

PHASE 2 — Database & Real Persistence

Goal: All data survives closing and reopening the app.
Investment: $2,500
Timeline: Week 2

Cursor Prompt — Phase 2

```
Phase 1 is complete. The Tauri app launches with all 8 modules.
Now implement Phase 2: real SQLite persistence using tauri-plugin-sql.
```

RULES:
- Use tauri-plugin-sql v2 ONLY (never better-sqlite3)
- Every database operation goes through src/services/db.ts
- Every recommendation written to audit_log table
- Use the schema exactly as defined in PRODUCTION_ROADMAP.md

TASKS:

1. Create src/services/db.ts:
   - Singleton database connection using invoke('initialize_database')
   - Export typed query helper functions
   - All queries return typed TypeScript objects

2. Create src/services/clientService.ts with these functions:
   - getAllClients(): Promise<Client[]>
   - getClient(id: string): Promise<Client>
   - createClient(data: Partial<Client>): Promise<Client>
   - updateClient(id: string, data: Partial<Client>): Promise<Client>
   - deleteClient(id: string): Promise<void>
   - generateId(): string (use crypto.randomUUID())

3. Create src/services/auditService.ts:
   - logEntry(type, clientId, input, output, reasoning, model): Promise<void>
   - getLog(limit?: number): Promise<AuditEntry[]>
   - IMPORTANT: every recommendation MUST call logEntry before returning

4. Create src/services/searchService.ts:
   - searchKnowledge(query: string, stage?: string): Promise<SearchResult[]>
   - Uses FTS5 virtual table via invoke('search_knowledge')

5. Create src/types/client.ts, document.ts, audit.ts with full TypeScript interfaces

6. Add these Tauri commands to main.rs and wire to database.rs:
   - get_all_clients
   - get_client(id)
   - create_client(data)
   - update_client(id, data)
   - delete_client(id)
   - search_knowledge(query, stage)
   - log_audit_entry(type, client_id, input, output, reasoning, model)
   - get_audit_log(limit)

7. Port these modules to use real data (replace sample data):
   - ExecutiveDashboard.tsx — read live counts from SQLite

```
   - ClientIntelligence.tsx — full CRUD using clientService
   - AuditTransparency.tsx — read from audit_log table

8. Keep these modules on sample data for now (Phase 3 wires them):
   - PipelineVisualizer.tsx
   - LiveCoachingAssistant.tsx
   - PostCallAnalysis.tsx
   - AdminStreamliner.tsx

ACCEPTANCE TEST:
- Create a new client in ClientIntelligence
- Close the app completely
- Reopen — client still exists
- AuditTransparency shows the create action logged
- Search in CoachingAssistant returns knowledge base results
```

Phase 2 Acceptance Checklist

```
[ ] Create client → close app → reopen → client still there
[ ] Client list loads from SQLite on startup
[ ] Audit log shows every action taken
[ ] FTS5 search returns knowledge base results
[ ] No sample data hardcoded in Dashboard or ClientIntelligence
[ ] All TypeScript types defined and used
[ ] Zero any types (strict TypeScript)
```

PHASE 3 — Document Ingestion Pipeline

Goal: Drop a PDF into a folder. Client profile appears automatically.
Investment: $3,000
Timeline: Week 2-3

Pre-Phase Requirement

Before writing any code: Get 5 DISC PDFs and 3 You 2.0 reports from Sandy. Run this test in isolation:

```bash
cd src-tauri
cargo test pdf_extraction -- --nocapture
```

Confirm text extraction works on real files before building the pipeline around it.

Cursor Prompt — Phase 3

```
Phase 2 is complete. SQLite persistence is working.
Now implement Phase 3: automatic document ingestion pipeline.

ARCHITECTURE:
File dropped into watched folder
  → file_watcher.rs fires event
  → Rust extracts raw text (pdf-extract → lopdf fallback)
  → Raw text sent to Ollama LLM for structured extraction
  → LLM returns JSON
  → TypeScript validates JSON schema
  → Valid data written to SQLite
  → Invalid data flagged in UI for manual review

WATCHED FOLDERS (create on app startup if they don't exist):
~/SandiBot/clients/DISC/
~/SandiBot/clients/You2/
~/SandiBot/clients/Fathom/
~/SandiBot/clients/Vision/

TASKS:

1. Update file_watcher.rs:
   - Start watching all four folders automatically on app launch
   - Determine document type from subfolder name
   - Emit 'file-detected' event with {path, name, type, folder}
   - Debounce: ignore duplicate events within 2 seconds

2. Create src-tauri/src/ollama.rs:
   - HTTP client calling localhost:11434
   - Function: parse_document(text: String, doc_type: String) -> Result<Value>
   - 30 second timeout
   - Returns raw LLM JSON response
   - If Ollama unreachable: return Err with "ollama_unavailable" code
   - Fallback is handled in TypeScript layer

3. Create prompts/ folder with prompt templates:
   - disc.txt — extract name, D score, I score, S score, C score,
                dominant style, key tendencies
   - you2.txt — extract dangers list, opportunities list, skills
```

- fathom.txt — extract summary, objections, commitments, next step
- vision.txt — extract vision statement, goals, timeline

Each prompt must end with:
"Return ONLY valid JSON with no explanation, no markdown, no code blocks."

4. Create src/services/ollamaService.ts:
- parseDocument(text, docType): Promise<ParsedDocument>
- Uses invoke('parse_with_llm', {text, docType, promptTemplate})
- Falls back to regex extraction if Ollama unavailable
- Validates response against Zod schema before returning

5. Create src/services/documentService.ts:
- validateParsedDocument(data, docType): ValidationResult
- saveDocument(clientId, filename, docType, extractedData): Promise<void>
- importBulk(files: FileList): Promise<BulkImportResult>

6. Add Tauri commands to main.rs:
- parse_with_llm(text, doc_type, prompt)
- check_ollama() → returns true/false
- get_file_watcher_status()
- bulk_import_directory(path)

7. Create validation schemas (Zod) in src/types/:
- discSchema — d,i,s,c all numbers 0-100, name string
- you2Schema — dangers and opportunities arrays
- fathomSchema — summary string, arrays for objections/commitments
- visionSchema — vision string

8. Create bulk import UI in AdminStreamliner.tsx:
- "Import Existing Files" button
- Folder picker using tauri-plugin-dialog
- Progress bar showing files processed
- Results summary: X succeeded, Y failed
- Failed files listed for manual review

9. Create outcome labeling UI in AdminStreamliner.tsx:
- List all clients without outcome labels
- For each: ACTIVE / CONVERTED / STALLED / CLOSED buttons
- Save outcome to clients.outcome field in SQLite
- This is Sandy's one-hour session to label 60 existing clients

ACCEPTANCE TEST:
- Drop real DISC PDF in ~/SandiBot/clients/DISC/

```
- Client profile appears in dashboard within 30 seconds
- DISC scores correctly extracted and shown
- Action logged in audit trail with extraction source noted
- Drop invalid file — app flags it, does not crash
- Bulk import 5 test files — all succeed
- Outcome labeling session: label 10 clients in under 5 minutes
```

Phase 3 Acceptance Checklist

```
[ ] File watcher starts automatically on app launch
[ ] Drop DISC PDF → profile appears in under 30 seconds
[ ] DISC scores match what is in the actual document
[ ] Failed imports show in UI with reason
[ ] Bulk import processes all files in a folder
[ ] Outcome labeling UI works and saves to database
[ ] Ollama offline → graceful fallback, no crash
[ ] All extractions logged in audit trail
```

PHASE 4 — Backup, Polish & Installer

Goal: Production-ready app. Sandy can install and use it daily.
Investment: $2,500
Timeline: Week 4

Cursor Prompt — Phase 4

```
Phase 3 is complete. Document ingestion pipeline works.
Now implement Phase 4: backup system, UI polish, and installer.

TASKS:

1. Fix backup.rs (apply validation fixes from PRODUCTION_ROADMAP.md):
   - Replace Builder::default().build() with Database::load()
   - Try backup locations in order:
     1. External drive D:/SandiBot-Backup (Windows)
     2. /Volumes/SandiBot-Backup (Mac external)
     3. ~/SandiBot/backups (local fallback, always works)
   - Trigger backup on app close
   - Trigger backup daily if app left open

2. Create src/components/StatusBar.tsx:
```

- Shows "Last backup: X days ago" — red if over 7 days
- Shows Ollama status: green dot Running / grey dot Offline
- Shows database record count
- Appears at bottom of every screen

3. Create backup restore flow in AdminStreamliner.tsx:
   - List available backup files with dates
   - Restore button with confirmation dialog
   - Progress indicator during restore
   - Restart prompt after restore completes

4. Error boundaries — wrap every module:
   Create src/components/ErrorBoundary.tsx if not already done:
   - Catches React render errors per module
   - Shows "This module encountered an error" card
   - Shows error details in collapsed section
   - Other modules continue working normally
   - Error automatically logged to audit_log

5. App icons — generate all required sizes:
   Create icons in src-tauri/icons/:
   - 32x32.png
   - 128x128.png
   - 128x128@2x.png
   - icon.icns (Mac)
   - icon.ico (Windows)
   Use Dr. Data mascot or simple DB icon if mascot not available.

6. Loading states — add to all async operations:
   - Skeleton loader while client list loads
   - Spinner on file import
   - Progress bar on bulk import
   - Disabled state on buttons during async operations

7. Empty states — add to all list views:
   - "No clients yet. Import your first client file." with import button
   - "No sessions logged yet." with log session button
   - "No backups found." with backup now button

8. Windows installer test:
   npm run tauri:build
   Confirm output in src-tauri/target/release/bundle/
   Document exact SmartScreen warning message and workaround steps

```
ACCEPTANCE TEST:
- Fresh install on machine with no dev tools
- All 8 modules work on installed version
- Backup creates file and status shows correctly
- Restore from backup works and data is intact
- Crash one module — others keep working
- App passes 3-day UAT with Sandy using real files
```

Phase 4 Acceptance Checklist

```
[ ] Backup runs automatically and status shows in StatusBar
[ ] Backup file exists at expected location after running
[ ] Restore from backup recovers all client data
[ ] Every module has ErrorBoundary — test by throwing error
[ ] Loading states show during all async operations
[ ] Empty states show for new installation
[ ] npm run tauri:build produces .exe and .dmg
[ ] App installs on machine with no Rust or Node installed
[ ] SmartScreen warning documented with workaround
[ ] Sandy completes 3-day UAT with real client files
```

PHASE 5 — Neo4j Knowledge Graph

Goal: Client relationships, patterns, and history mapped in graph database.
Investment: Retainer Phase
Timeline: Month 2
Prerequisite: 50+ clients in SQLite with outcome labels

Cursor Prompt — Phase 5

```
Phases 1-4 are complete and delivered. Sandy has been using the app
for 4+ weeks and has 50+ clients with outcome labels in SQLite.
Now implement Phase 5: Neo4j local knowledge graph.

PREREQUISITES CHECK BEFORE STARTING:
- Neo4j Community installed at localhost:7687
- Sandy has 50+ clients in SQLite with outcome labels
- Sandy has 100+ sessions logged

GRAPH SCHEMA:
Nodes: Client, DISCProfile, Stage, Objection, Script, Outcome, Session
```

Relationships:
  (Client)-[HAS_PROFILE]->(DISCProfile)
  (Client)-[CURRENTLY_IN]->(Stage)
  (Client)-[RAISED]->(Objection)
  (Session)-[USED]->(Script)
  (Session)-[RESULTED_IN]->(Outcome)
  (Client)-[SIMILAR_TO]->(Client) -- computed by embedding distance later

TASKS:

1. Add Neo4j health check to app startup:
   - Check localhost:7687 is reachable
   - Show Neo4j status in StatusBar
   - App works fully without Neo4j (graceful degradation)

2. Create src/services/neo4jService.ts:
   - connect(): Promise<void>
   - runQuery(cypher, params): Promise<Record[]>
   - getSimilarClients(clientId): Promise<Client[]>
   - getStagePatterns(stage): Promise<Pattern[]>

3. Create migration script scripts/migrate_to_neo4j.ts:
   - Reads all clients from SQLite
   - Creates corresponding Neo4j nodes
   - Wires relationships from session history
   - Idempotent — safe to run multiple times

4. Add graph queries to dashboard:
   - "Clients similar to [current client]" widget
   - "What worked at this stage" suggestion panel
   - Each suggestion shows evidence: "Based on 3 similar clients"

5. Add to AuditTransparency module:
   - Show graph query used to generate each suggestion
   - Full evidence trail for every recommendation

ACCEPTANCE TEST:
- Migration script runs without errors
- All clients exist as nodes in Neo4j
- Similar client query returns 3+ meaningful matches
- Stage pattern query returns real historical data
- Every graph-based suggestion shows evidence trail

PHASE 6 — Intelligence Engine

Goal: System recommends next best actions based on real patterns.
Investment: Retainer Phase
Timeline: Month 3 (rules), Month 6 (graph patterns), Month 12+ (embeddings)

Phase 6A — Deterministic Rules Engine

```
Implement PUSH/NURTURE/PAUSE scoring using real SQLite data.

SCORING ALGORITHM (weighted):
  readiness_average    × 0.35   (avg of 4 readiness scores)
  days_since_contact   × 0.25   (inverse — longer = lower score)
  stage_velocity       × 0.20   (how fast moving through stages)
  pink_flag_count      × 0.20   (inverse — more flags = lower score)

Score 70-100 → PUSH
Score 40-69  → NURTURE
Score 0-39   → PAUSE

EVERY recommendation must:
1. Calculate score with all four components
2. Write to audit_log with full reasoning breakdown
3. Show Sandy the reasoning — not just the label

Implement in: src/services/recommendationService.ts
Surface in: ExecutiveDashboard.tsx and LiveCoachingAssistant.tsx
```

Phase 6B — Graph Pattern Recognition

Prerequisite: 50+ labeled outcomes in Neo4j graph

```
Cypher queries to implement:
1. Similar clients (same DISC type, same stage, similar readiness)
2. Objection frequency by DISC type at each stage
3. Script effectiveness — which scripts led to CONVERTED outcomes
4. Stage transition time — average days per stage by DISC type

Surface in dashboard as:
"Clients similar to Dave: Sarah (converted), Mike (converted), Tom (stalled)"
"At this stage with D profile: PUSH works 78% of the time"
```

Phase 6C — Embedding Predictions

Prerequisite: 200+ labeled outcomes in Neo4j graph

```
Training script: scripts/train_embeddings.py
  - Reads Neo4j graph
  - Trains Node2Vec embeddings
  - Saves model to ~/SandiBot/models/embeddings.pkl
  - Run manually, not automatically

Inference: Called via Tauri Python sidecar
  - Load saved model
  - Find top 3 similar clients by vector distance
  - Return confidence score and evidence
  - Display in dashboard as "Next Best Action" card
```

PHASE 7 — LLM Document Parsing Upgrade

Goal: Replace brittle regex parsing with intelligent LLM extraction.
Investment: Retainer Phase
Timeline: Month 6-8

Phase 3 shipped with regex as baseline.
Phase 7 upgrades all parsing to LLM-based extraction.

ARCHITECTURE:
pdf-extract → raw text → Ollama LLM → structured JSON → Zod validation → SQLite

MODEL ROUTING:
Simple structured docs (DISC scores) → phi3:mini (fast)
Complex narrative docs (You 2.0) → llama3.1:8b
Conversational transcripts (Fathom) → llama3.1:8b

PROMPT ENGINEERING RULES:
Always end with: "Return ONLY valid JSON. No explanation. No markdown."
Include example output in prompt
Include field names matching SQLite schema exactly
Include validation rules (e.g. "DISC scores must be integers 0-100")

FALLBACK CHAIN:
LLM available + valid JSON → use LLM result
LLM available + invalid JSON → log warning, try regex
LLM unavailable → use regex silently
Regex fails → flag for manual review

All parsing decisions logged to audit_log with model_used field.

PHASE 8 — Productization

Goal: Reusable system for additional coaches.
Investment: New client engagements
Timeline: Year 2 (after Sandy success documented)

```
Prerequisites:
  - Sandy using system successfully for 3+ months
  - Documented case study with real numbers
  - At least one referral ready to start

Deliverables:
  - config.json per coach (methodology, stage names, document types)
  - Onboarding wizard (first-launch setup)
  - Vertical ontology packages:
      franchise_coaching_v1.json
      executive_coaching_v1.json
  - White-label installer (custom name, color, icon)
  - Multi-coach architecture (separate DB per instance)

Pricing model for additional coaches:
  Setup:    $5,000-15,000 (configured, not custom)
  Monthly:  $500-1,500 retainer
  Enterprise (franchisor buying for all coaches): quote separately
```

Known Issues — Track and Fix

Issue	File	Priority
move || syntax fix	file_watcher.rs	P0 — Phase 1
dirs::home_dir() missing dep	file_watcher.rs	P0 — Phase 1
Database::load() pattern	backup.rs	P0 — Phase 1
Database::load() pattern	database.rs	P0 — Phase 1
lopdf raw bytes extraction	pdf_parser.rs	P0 — Phase 1
DISC regex too loose	pdf_parser.rs	P2 — Phase 3
App.tsx is test harness	App.tsx	P1 — Phase 1
index.css Inter font	index.css	P2 — Phase 1
App.css dead code	App.css	P1 — Phase 1
No error boundaries	—	P1 — Phase 1
tauri-plugin-sql Builder API	backup.rs	P0 — Phase 1

Retainer Support — What Changes Over Time

Month	Typical Work

1	Setup, onboarding, real-file testing, edge cases
2	First usage issues, PDF parsing tuning, Neo4j setup
3	Quiet. Graph data accumulating. Monitor.
4	Windows update compatibility check if needed
5	Quiet. Sandy asks about new features. Scope them.
6	Ollama model upgrade (better model available)
7	Quarterly backup audit — verify restore works
8	Graph pattern queries maturing — tune thresholds
9-11	Quiet. Retainer is passive.
12	Annual review. Year 2 roadmap. New change orders.

Hardware Requirements for Sandy

```
MINIMUM
  OS:       Windows 10/11 (64-bit) or macOS 12+
  RAM:      16GB
  Storage:  512GB SSD (not spinning drive)
  Port:     Thunderbolt 4 (for future eGPU option)

RECOMMENDED
  RAM:      32GB
  Storage:  1TB NVMe SSD
  CPU:      Intel i7 13th gen or AMD Ryzen 7

ONE-TIME INTERNET REQUIRED (setup only)
  Ollama installer:      ~200MB
  phi3:mini model:       ~2.3GB
  llama3.1:8b model:     ~4.7GB
  Neo4j Community:       ~500MB
  Total:                 ~8GB download, 20-30 minutes

AFTER SETUP: Zero internet required. Ever.
```

Delivery Checklist — What Sandy Receives

```
[ ] SandiBot-Setup.exe        Windows installer
[ ] SandiBot-Setup.dmg        Mac installer
[ ] UserGuide.pdf             Daily use instructions
```

[ ] BackupGuide.pdf	Backup and restore procedures
[ ] HardwareGuide.pdf	Requirements and upgrade path
[ ] SetupChecklist.pdf	One-time Ollama and Neo4j setup

This document is the single source of truth for all production development.
All Cursor prompts should reference this file.
Last updated: March 2026
Developer: Dr. Data — Decision Intelligence
drdatadecisionintelligence.com